Stylesheet conflicts

- As you add more and more CSS to a stylesheet, you will **inevitably run into conflicts**.
- How to resolve these conflicts?
  - Cascading
  - Specificity
  - Inheritance

Cascading

- Cascading refers to the logic your browser uses to determine which CSS rulesets are the most important, especially when they conflict with each other.
- One of the simplest ways your browser does this is by paying attention to the order in which your rulesets appear.

Cascading (cont.)

For example, imagine your stylesheet looked like this:

```
a { color: purple; }
a { color: green; }
```

Both rulesets are targeting the same HTML—all <a> elements on the page, but telling the browser to do two different things, and it can't do both at once.

Selector precedence /Specificity

- If there are two or more CSS rules that point to the same element, the selector with the highest specificity value will "win", and its style declaration will be applied to that HTML element.
- Think of specificity as a score/rank that determines which style declaration is ultimately applied to an element.

```
<style>
        p{color:red;}
 </style>

<p>Hello World!</p>
```

Specificity (cont.)

- In this example, we have added a class selector (named "test"), and specified a green color for this class. The text will now be green (even though we have specified a red color for the element selector "p"). This is because the class selector is given higher priority:

```
<style>
  .test {color: green;}
  p {color: red;}
</style>

<p class="test">Hello World!
</p>
```

Specificity (cont.)

- In this example, we have added the id selector (named "demo"). The text will now be blue, because the id selector is given higher priority:

```
<style>
    #demo {color: blue;}
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>

<p id="demo" class="test">H
ello World!</p>
```

Specificity (cont.)

- In this example, we have added an inline style for the "p" element. The text will now be pink, because the inline style is given the highest priority:

```
<style>
  #demo {color: blue;}
  .test {color: green;}
  p {color: red;}
</style>

<p id="demo" class="test" style="color: pink;">Hello World!</p>
```

Specificity Hierarchy

- Every CSS selector has its place in the specificity hierarchy.
- There are four categories which define the specificity level of a selector:
- **Inline styles** - Example: <h1 style="color: pink;">
- **IDs** - Example: #navbar
- **Classes, pseudo-classes, attribute selectors** - Example: .test, :hover, [href]
- **Elements and pseudo-elements** - Example: h1, ::before

Calculating specificity

Selector	Specificity Value	Calculation
p	1	1
P .test	11	1 + 10
P #demo	101	1 + 100
<p style="color: pink;">	1000	1000
#demo	100	100
.test	10	10
P .test1 .test2	21	1 + 10 + 10
#navbar p #demo	201	100 + 1 + 100

```
A: h1
B: h1 #content
C: <h1 id="content" style=
"color:
pink;">Heading</h1>
```

Inheritance

- Inheritance allows child elements to inherit certain styles from their parent element, which helps reduce redundant code and ensures consistent styling across related elements.

```
<div class="parent">
    <h1>Title</h1>
    <p>This is a paragraph of text. <span>And this is a span inside a paragraph.</span></p>
</div>
```

```
.parent {
    color: dark green;
    font-family: Arial, sans-serif;
}

h1 {
    text-transform: uppercase;
}

span {
    color: dark red;
}
```

Default Browser Styles

- Browsers have default CSS styles
  - Used when there is no CSS information or any other style information in the document
  - Silently inherited in all documents
- Caution: default styles differ in browsers
  - E.g. Firefox default page background is white, while IE7 uses about 5% gray background

Linking HTML and CSS

- HTML (content) and CSS (presentation) can be linked in three ways:
  - Inline: the CSS rules in the `style` attribute
    - No selectors are needed
  - Embedded: in the HTML `<style>` tag
  - External: CSS rules are in a separate file
    - Usually a file with a `.css` extension
    - Linked via `<link rel="stylesheet" href=…>` tag

Linking HTML and CSS (2)

- Inline styles have highest priority
  - Then are the embedded styles
  - External styles are last
- Using external files is highly recommended
  - Simplify the HTML document
  - Benefit from browser's cache
- Inline styles are about to be deprecated by the W3C

Inline Styles

- Inline CSS styles
  - Individual element's style defined using `style` attribute
  - Contains only declaration, no selectors:

```
<p style="font-size:20pt; color:
#0000FF">
```

  - Override any other styles
  - Apply to all descendant elements
  - Used for styles that are not needed anywhere else in the Web site

Inline Styles: Example

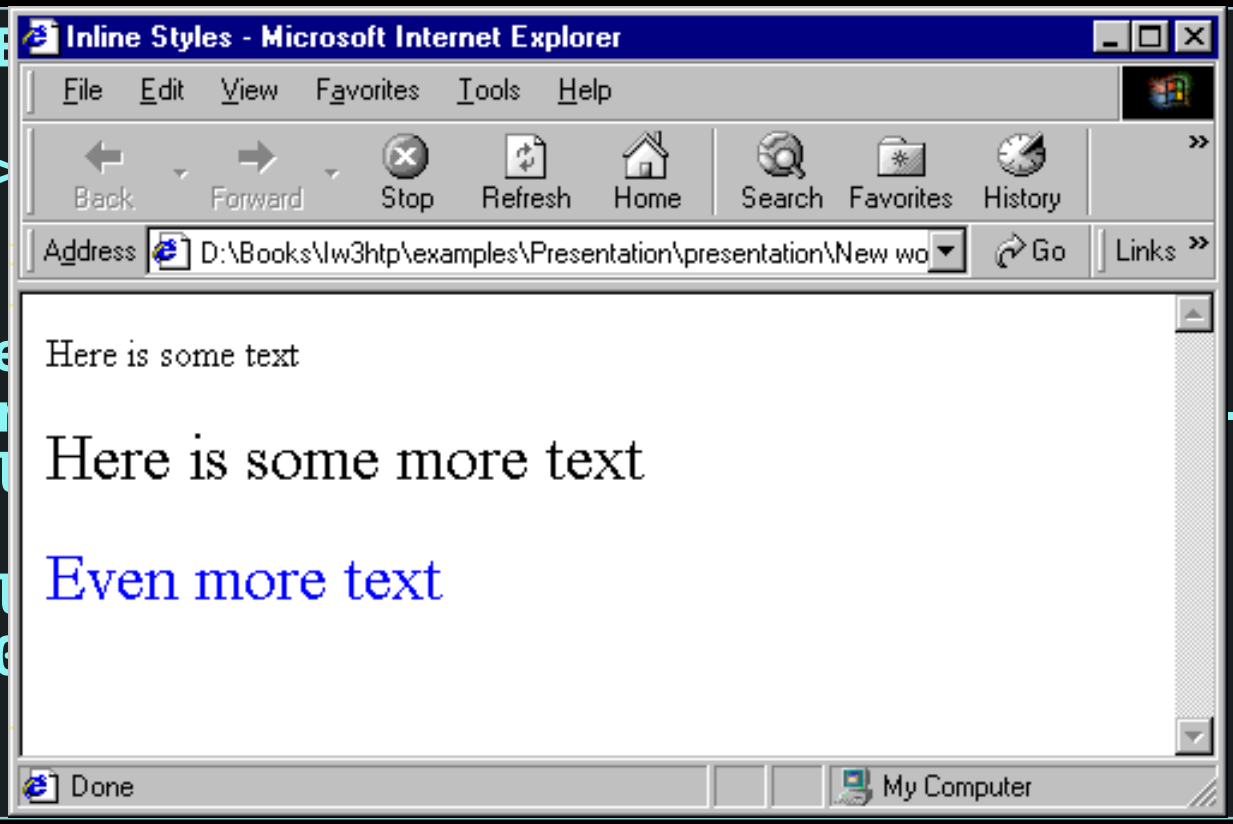
inline-styles.html

```
<!DOCTYPE html>
<head>
  <title>Inline Styles</title>
</head>
<body>
  <p>Here is some text</p>
<!--Separate multiple styles with a semicolon-->
  <p style="font-size: 20pt">Here is some
    more text</p>
  <p style="font-size: 20pt;color:
    #0000FF" >Even more text</p>
</body>
</html>
```

Inline Styles: Example

inline-styles.html

```
<!DOCTYPE
<head>
  <title>
</head>
<body>
  <p>Here
<!--Separ                                    -->
  <p styl
    more
  <p styl
    #0000
</body>
</html>
```



Embedded Styles/Internal styling

- Embedded in the HTML in the <style> tag:

```
<style>
      {

      }
</style>
```

  - The <style> tag is placed in the <head> section of the document
  - Styles apply to the whole document

External CSS Styles

- External linking
  - Separate pages can all use shared style sheet
  - Only modify a single file to change the styles across your entire Web site
- `link` tag (with `rel` attribute)
  - Specifies a relationship between current document and another document

```
<link rel="stylesheet" href="styles.css">
```

  - `link` element can stay only in the HTML

External Styles: Example

styles.css

```
/* CSS Document */

a       { text-decoration: none }

a:hover { text-decoration: underline;
          color: red;
          background-color: #CCFFCC }

li em   { color: red;
          font-weight: bold }

ul      { margin-left: 2cm }

ul ul   { text-decoration: underline;
          margin-left: .5cm }
```

Values in the CSS Rules

- Colors are specified in RGB format, in hex form:
  - Example: `#A0A6AA`
  - Predefined color aliases exist: `black`, `blue`, etc.
- Numeric values are specified in:
  - Pixels, e.g. `12px`
  - Points, e.g. `10pt`
  - Inches, centimeters, millimeters
    - E.g. `1in`, `1cm`, `1mm`



AMERICANA
AUSTERE
AVANT GARDE
BOOKMAN
COCHIN
COLLEGE
COOPER
DOROVAR
FRIZ QUAD
Garamond
KABEL
Fraktur
Palatino
RALEIGH
ROOST
Tiffany
TIMES

Ambiente
Amaze
Becka
Bible Script
Brush Script
Candoo
Caflish
Churchward
Dianna
Flamenco
Indy Script
Kaiser
Palette
Scrawlin
Sentimental
Zephen Script



Fonts



font-rules.html



CSS Rules for Fonts

- `color` – specifies the color of the text
- `font-size` – size of font: `xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, `xx-large`, `smaller`, `larger` or numeric value
- `font-family` – comma separated font names
  - Example: `Verdana`, `Sans-Serif`, etc.
  - The browser loads the first one that is available
  - There should always be at least one serif font

CSS Rules for Fonts (2)

- `font-style` – styles the font
  - Values: `normal`, `italic`, `oblique`
- `text-decoration` – decorates the text
  - Values: `none`, `underline`, `line-trough`, `overline`, `blink`
- `text-align` – defines the alignment of text or other content
  - Values: `left`, `right`, `center`, `justify`

Short Font Rules

- `font`
  - Shorthand rule for setting multiple font properties at the same time

```
font: italic normal bold 12px Verdana;
```

is equal to writing this:

```
font-style: italic;
font-variant: normal;
font-weight: bold;
font-size:12px;
font-family: Verdana;
```

Background CSS Rules

- background-color
  - Solid color background
- background-image
  - URL of image to be used as background, e.g.:

```
background-image:url("back.gif");
```

- background-repeat
  - repeat-x, repeat-y, repeat, no-repeat
- background-attachment
  - fixed / scroll

Background CSS Rules (2)

- `background-position`: specifies vertical and horizontal position of the background image
  - Vertical position: `top`, `center`, `bottom`
  - Horizontal position: `left`, `center`, `right`
  - Both can be specified in percentage or other numerical values
  - Examples:

```
background-position: top left;
```

```
background-position: -5px 50%;
```

Background Short Rule

- background: shorthand rule for setting background properties at the same time:

```
background: #FFF0C0 url("back.gif") no-repeat fixed top;
```

is equal to writing:

```
background-color: #FFF0C0;
background-image: url("back.gif");
background-repeat: no-repeat;
background-attachment: fixed;
background-position: top;
```

- Some browsers will not apply BOTH color and image for background if using shorthand rule


Borders
Live Demo
border-rules.html

Border CSS Rules

- `border-width`: `thin`, `medium`, `thick` or numerical value (e.g. `10px`)
- `border-color`: color alias or RGB value
- `border-style`: `none`, `hidden`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, `outset`
- Each property can be defined separately for left, top, bottom and right
  - `border-top-style`, `border-left-color`, …

Border Short Rules

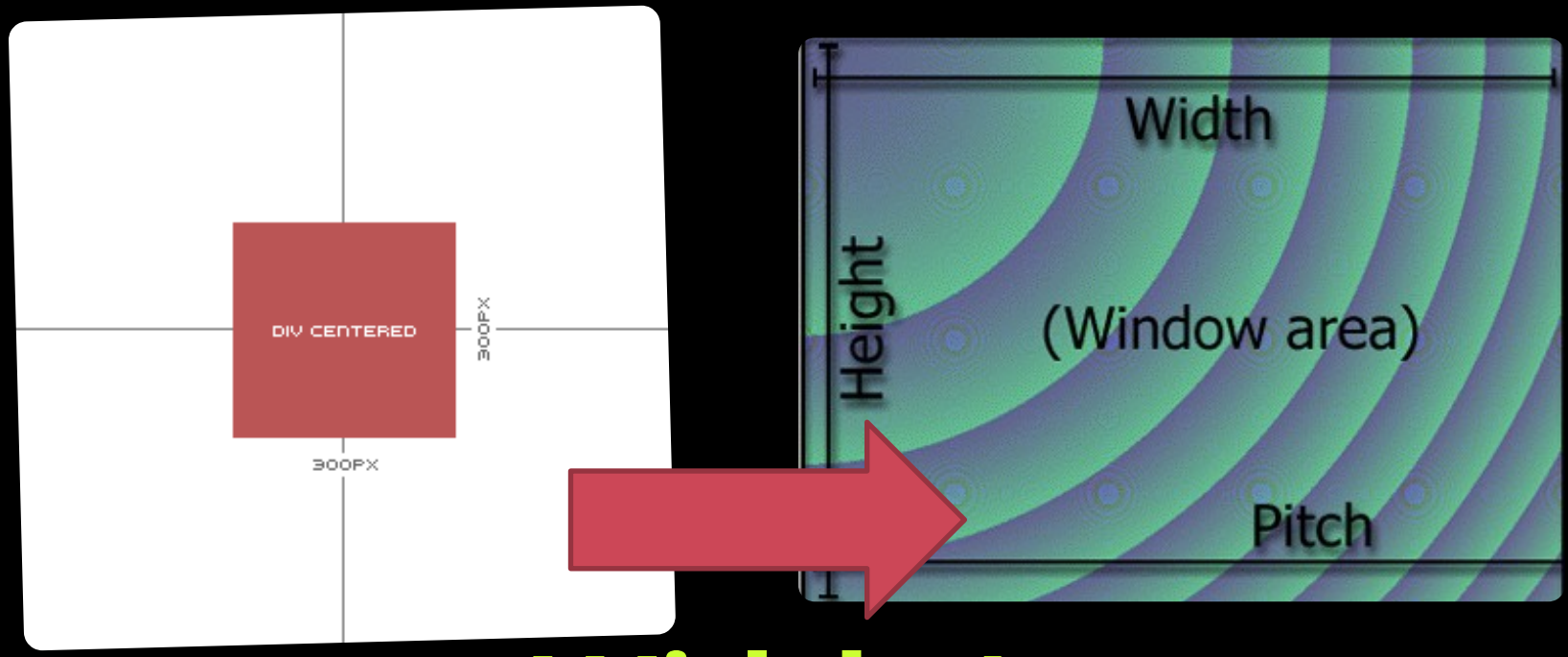
- `border`: shorthand rule for setting border properties at once:

```
border: 1px solid red;
```

  is equal to writing:

```
border-width:1px;
border-color:red;
border-style:solid;
```

- Specify different borders for the sides via shorthand rules: `border-top`, `border-left`, `border-right`, `border-bottom`



Width / Height

size-rules.html

Width, Height CSS Rules

- `width` – defines numerical value for the width of element, e.g. `200px`
- `height` – defines numerical value for the height of element, e.g. `100px`
  - Important: not all elements and browsers follow this value!
    - Usually the height of an element is defined by its content
    - Common mistake is to apply height to tables or table cells



Margins and Paddings

Live Demo

margins-paddings-rules.html

The Box Model



Margin and Padding

- `margin` and `padding` define the spacing around the element
  - Numerical value, e.g. `10px`
  - Can be defined for each of the four sides separately
    - `margin-top`, `padding-left`, …
  - `margin` is the spacing outside of the border
  - `padding` is the spacing between the border and the content

Margin and Padding: Short Rules

- `margin: 5px;`
  - Sets all four sides to have margin of 5 px;
- `margin: 10px 20px;`
  - Sets margins: top and bottom to 10px, left and right to 20px;
- `margin: 1px 3px 5px 7px;`
  - Sets top, right, bottom, left margins to 1px, 3px, 5px and 7px respectively
- Same shorthand rules apply for `padding`

CSS Rules for Positioning

- `position`: defines the positioning of the element, depending on the parent elements
- The value is one of:
  - `static` (default)
  - `relative` – relative position according to where the element would appear with static position
  - `absolute` – position according to the parent element

CSS Rules for Positioning

- `top`, `left`, `bottom`, `right`: specifies offset of absolute/fixed/relative positioned element as numerical values
- `vertical-align`: sets the vertical-alignment of an element
  - Usually used for table cells
  - Values: `baseline`, `sub`, `super`, `top`, `text-top`, `middle`, `bottom`, `text-bottom` or numeric
- `z-index`: specifies the depth-order of element

CSS Rules for Positioning

Vertical Align Example

Baseline Align 

Top Align 

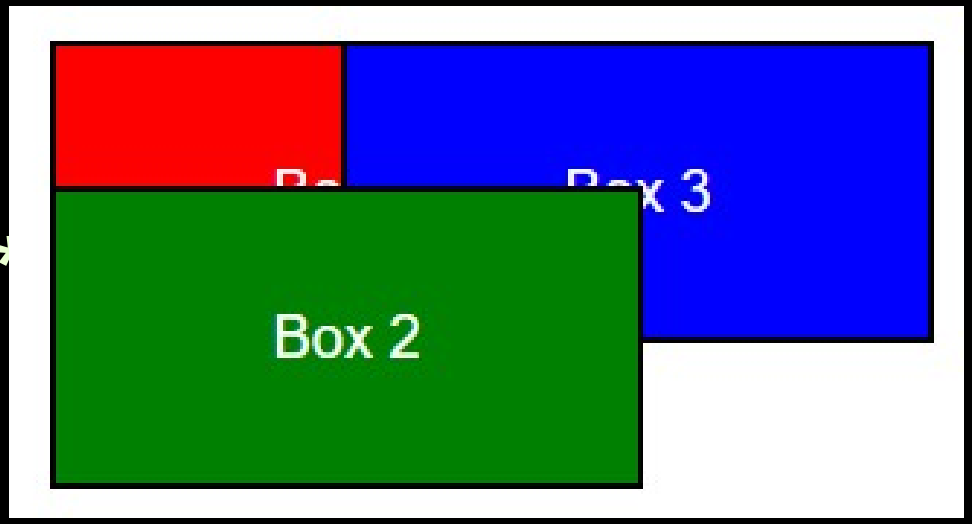
Middle Align 

Bottom Align 

Table Example

Top Aligned	Middle Aligned	Bottom Aligned

CSS Rules for Positioning



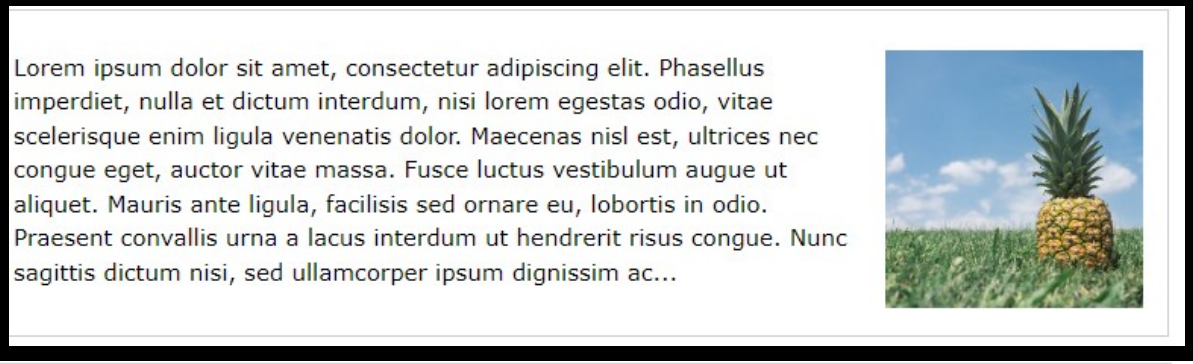

```
.box1 {
z-index: 1; /* Lower z-index *
}
.box2 {
 z-index: 3; /* Higher z-index, so it appears above box1 */
}
.box3 {
z-index: 2; /* Middle z-index, so it appears between box1 and box2 */
}
```

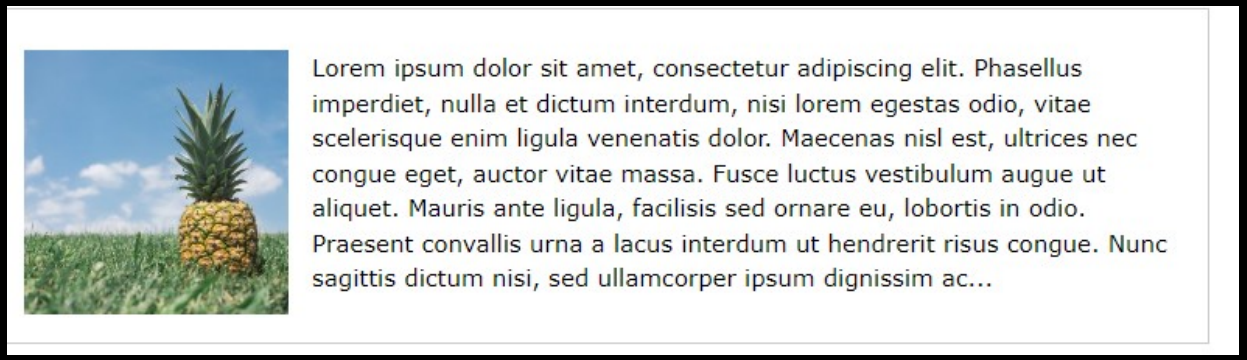
Floating Elements

float-rules.html

Float CSS Rule

- `float`: the element "floats" above the elements
  - `left`: places the element on the left
  - `right`: places the element on the right

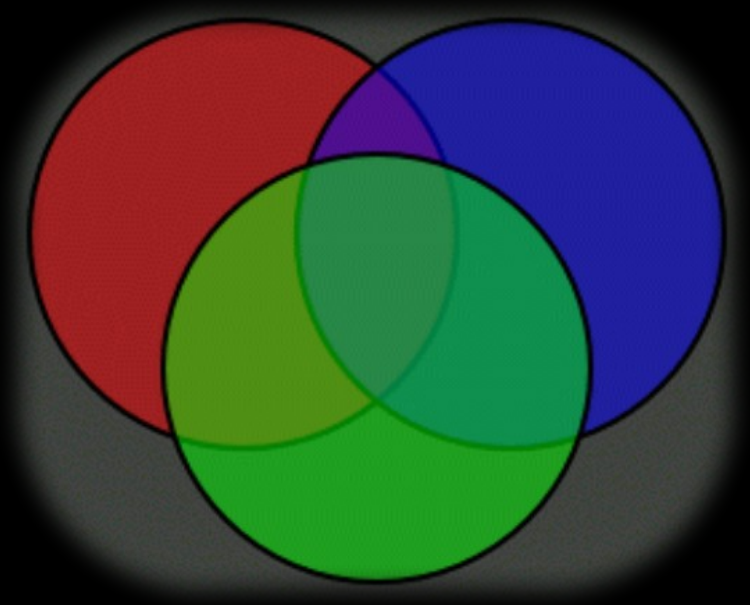




Clear CSS Rule

- `clear`
  - **Sets the sides of the element where other floating elements are NOT allowed**
  - **Possible values: `left`, `right`, `both`**
  - **This rule can be applied only to “floating” elements**

clear: left; clear: right; clear: both;

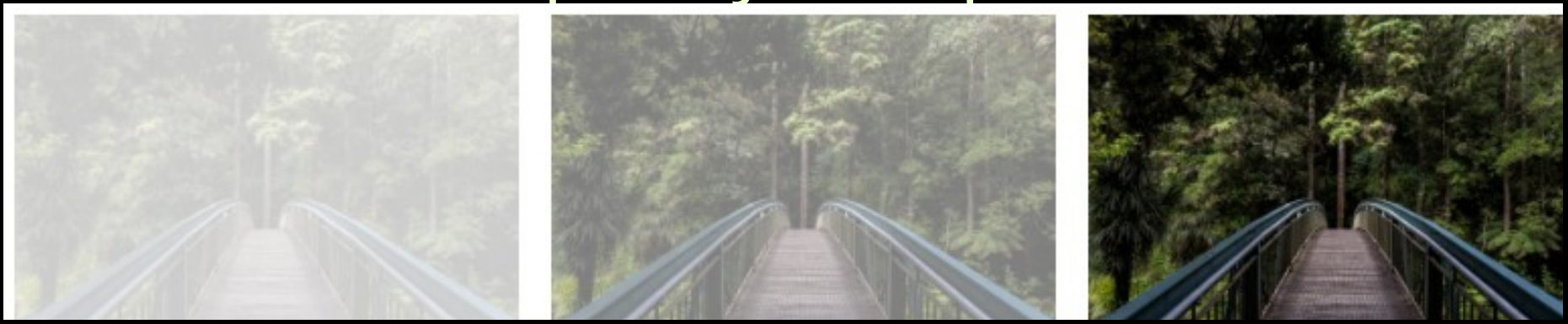


Opacity

opacity-rule.html

Opacity CSS Rule

- The opacity property sets the opacity level for an element.
- The opacity-level describes the transparency-level, where 1 is not transparent at all, 0.5 is 50% see-through, and 0 is completely transparent.



opacity 0.2 opacity 0.5 opacity 1(default)

Visibility

visibility-rule.html

Visibility CSS Rule

- `visibility`
  - Determines whether the element is visible or not.
  - `hidden`: element is not rendered, but still takes place in the rendering (acts like `opacity:0`)
  - `visible`: element is rendered normally

Display



display-rule.html

Display CSS Rule

- `display`: controls the display of the element and the way it is rendered and if breaks should be placed before and after the element.
- Block
- Inline
- Inline-block
- None

Display:Block CSS Rule

- **The element is displayed as a block-level element, meaning it takes up the full width available, and a new line is started before and after the element.**

```
img { display: block; }
```

Display: Inline CSS Rule

- **The element is displayed as an inline element, meaning it does not start on a new line and only takes up as much width as necessary**

```
p { display: inline; }
```

Display: Inline-block CSS Rule

- **The element behaves like an inline element (does not start on a new line) but has the ability to have its width and height adjusted, similar to block-level elements.**

```
.box {
        display: inline-block;
        width:      150px; height:
        100px;      margin: 10px;
        padding:    20px; border: 1px
        solid  black;
}
```

Display: None CSS Rule

- The element is completely removed from the document flow, as if it does not exist. It is neither displayed nor does it take up any space on the page.

Overflow

Live Demo

overflow-rule.html

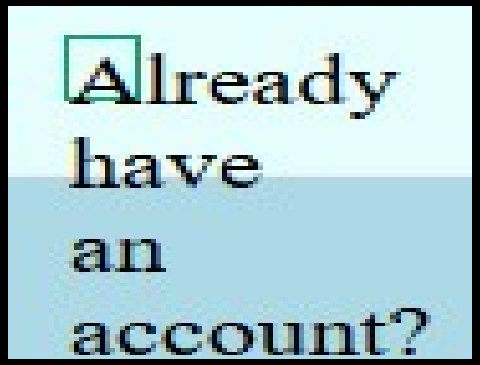


Overflow CSS Rule

- The CSS overflow property controls what happens when the content of an element is too large to fit within its container. It is particularly useful for handling overflow in block-level elements, such as , when the content exceeds the specified height or width of the container.
- Visible
- Hidden
- Scroll

Visible

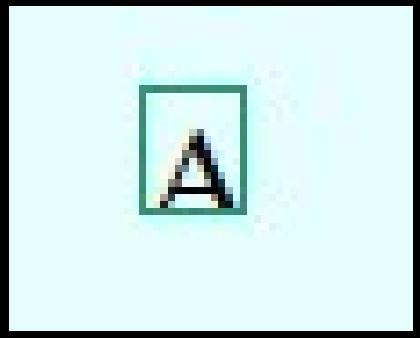
```
.para{
        width: 12px;
        height: 15px;
        border: #04AA6D solid 1px;
            overflow: visible;
    }
<div class="para">
            Already have an account?
 </div>
```



telerik

Hidden

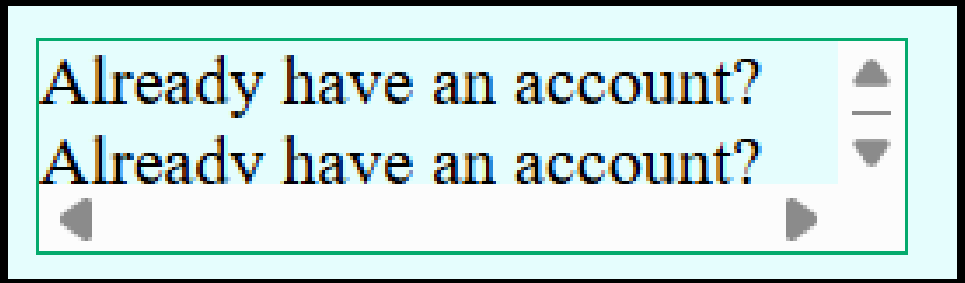
```
.para{
        width: 12px;
        height: 15px;
        border: #04AA6D solid
1px;
        overflow: hidden;
    }
<div class="para">
                Already have an
account?
 </div>
```



telerik

Scroll

```
.para{
        width: 12px;
        height: 15px;
        border: #04AA6D solid
1px;
        overflow: scroll;
    }
<div class="para">
            Already have an
account?
 </div>
```



Benefits of using CSS

- More powerful formatting than using presentation tags
- Your pages load faster, because browsers cache the `.css` files
- Increased accessibility, because rules can be defined according given media
- Pages are easier to maintain and update

What to Avoid

- Don’t use `<font>` or other presentation tags to change the look of your text
  - `<font color=...>`
  - `<b>text that is bold`
  - `<center> this text is centered`
- Do not use complex CSS rules-sets because may slow down page rendering
- Move as much as possible to external files
- Avoid inline CSS

telerik

box-sizing (content-box vs. border-box); width/height, min/max constraints and overflow